PROJECT BASED LEARNING (PBL)
DATA STRUCTURES AND ALGORITHMS (25CS2103E)
PROJECT DOCUMENTATION

Project Review - 2 Report

Project Title	SportsPulse: High-Performance Algorithmic Sports Analytics Platform
Use Case	Multi-Pattern Keyword Detection, Suffix Indexing, Dynamic Roster Optimization & Interactive Analytics
Programme / Year	[CSE-1-PBL] - II Year
Section	1
Academic Year	[2026-27]
Faculty / Mentor	Dr. Maganti Syamala

Team Members & Detailed Role Allocation

S.No.	Student Name	Roll No.	Individual Role / Module Contribution
1	**Jeshwanth Gummadavelly**	2500032042	Module 1 (String Matching & Aho-Corasick), Module 2 (Suffix Structures & Kasai LCP), Module 3 (Levenshtein DP & Bitmask Lineup Selection)
2	**Bhanu Teja M**	2500090202	Module 4 (Network Flow & Bipartite Role Matching), Module 5 (NP-Completeness & 2-Approx Vertex Cover), Module 6 (Randomized & ForkJoin Parallel Reduce)
3	**R Sai Kiran**	2500031571	Interactive Analytics Dashboard, Dataset Ingestion Pipeline, Visual Query Interface, System Integration & Testing

1. Updated Problem Statement and Objectives

1.1 Problem Statement

Modern professional sports organizations, analytics firms, and sports broadcast networks generate vast volumes of continuous, heterogeneous match data. In premier cricket tournaments such as the Indian Premier League (IPL), every ball bowled produces immediate telemetry: velocity, coordinates, boundary classifications, dismissal tags, physical fatigue tracking, and simultaneous unstructured commentary. As match archives and telemetry archives expand into tens of gigabytes, basic linear scanning and naive storage mechanisms encounter catastrophic processing bottlenecks.

Linear search routines cause substantial broadcast latency during live match-event detection; raw text sports reports remain unindexed and inaccessible for real-time tactical queries; user queries containing spelling mistakes or regional phonetic variations fail to match official registries; and combinatorial challenges such as optimal XI roster selection subject to synergy and budget constraints prove computationally intractable (NP-hard).

The objective of SportsPulse is to architect and implement an end-to-end, high-performance sports performance analytics platform in Java. The system directly maps critical sports operations to six core DSA paradigms: (1) Exact String Matching (KMP, Z-Algorithm, Rabin-Karp, Aho-Corasick Trie); (2) Suffix Structures (Suffix Array, Kasai's LCP); (3) Advanced Dynamic Programming (Levenshtein Distance, Bitmask Selection, Matrix Chain Multiplication); (4) Network Flow & Bipartite Matching (Edmonds-Karp, tactical fielder assignment); (5) NP-Completeness & Constraint Modeling (2-Approximation Vertex Cover, Clique synergy); and (6) Randomized & Parallel Algorithms (Randomized QuickSort, Reservoir Sampling, ForkJoin Parallel Reduction). An interactive analytics dashboard connects these back-end algorithms to real-time visualizations for team strategists and scouts.

1.2 Objectives

- To build a comprehensive sports intelligence architecture mapping sports operational challenges to verified Data Structures and Algorithms (CO1, CO2, CO3).
- To achieve linear-time multi-pattern event extraction across continuous match commentary streams using an Aho-Corasick finite state automaton (Module 1).
- To index historical IPL match reports and commentary logs using Suffix Arrays and Kasai's Longest Common Prefix (LCP) algorithm for rapid substring lookup (Module 2).
- To implement dynamic programming name auto-correction (Levenshtein Distance) and solve exponential roster selection under salary and role constraints via Bitmask DP (Module 3).
- To allocate fielding positions optimally and evaluate stadium gate throughput using Edmonds-Karp Network Flow and Maximum Bipartite Matching (Module 4).
- To model squad friction and compatibility as graph optimization problems, implementing a polynomial-time 2-Approximation Vertex Cover for conflict resolution (Module 5).
- To accelerate high-volume ball record aggregations across CPU cores via ForkJoin Parallel Reduce and extract unbiased live streaming deliveries via Reservoir Sampling (Module 6).
- To deliver an interactive, data-driven Analytics Dashboard connecting underlying algorithmic routines to actionable visual metrics for coaches and analysts.

1.3 Scope of the Project

The project encompasses algorithmic design, complexity analysis, Java-based implementation, and empirical verification using real-world Indian Premier League (IPL) historical data files: ball.csv, bat.csv,

and match.csv. The platform addresses raw CSV data streaming, memory optimization, multi-threaded parallel execution, and front-facing dashboard analytics. The system operates as an offline and simulated live-event analytical engine, providing modular components suitable for integration into enterprise broadcast graphics and team management infrastructure.

2. Final System Architecture

2.1 Architecture Explanation

The SportsPulse architecture follows a cohesive five-layer paradigm designed for low-latency processing, robust memory safety, and expressive visual analytics:

- **Data Ingestion & Parsing Tier:** Ingests raw IPL tournament datasets (ball.csv, bat.csv, match.csv) and live commentary streams, transforming raw CSV text into strongly-typed object instances (BallRecord, BatRecord, MatchRecord).
- **In-Memory Modeling Tier:** Organizes player, team, and delivery statistics into high-efficiency data structures including character tries, suffix arrays, flow networks, and bipartite adjacency lists.
- **Core DSA Algorithmic Processing Engine:** Houses the six specialized computational modules executing single/multi-pattern string matching, suffix phrase indexing, dynamic programming optimizations, network flow distributions, constraint approximations, and parallel reductions.
- **Decision & Analytics Processing Tier:** Synthesizes algorithmic outputs into tactical sports metrics: boundary/wicket milestone alerts, fuzzy search disambiguation, optimal fielding rosters, and cumulative batting averages.
- **Interactive Analytics Dashboard Tier:** The front-facing visual interface designed to accept analyst queries, render real-time event alerts, plot player performance trajectories, and display visual roster recommendations.

[INSERT FINAL SYSTEM ARCHITECTURE DIAGRAM HERE]
(Flow: CSV Datasets -> Ingestion Engine -> 6 DSA Modules -> Analytics Engine -> Interactive Dashboard)

2.2 Technology Stack

Category	Technology / Tool	Purpose in SportsPulse	Version
Programming Language	Java (JDK)	Core algorithmic implementations and multithreading	SE 17 / 21
Operating System	Windows 11 / Linux	Development, execution, and benchmarking environment	Windows 11
Data Processing Engine	Java ForkJoin & Collections	Parallel reduction trees, tries, custom graph adjacency	Native JVM
Frontend / Interface	Interactive Dashboard	Visual analytics display, query input, metrics rendering	v1.0 (GUI/CLI)
Build & Runtime Tool	Javac / Java CLI & Batch	Compilation, execution scripts, automated test suite	v21.0.x
Primary Dataset	IPL Cricket Historical CSVs	ball.csv, bat.csv, match.csv delivery and match telemetry	2008-2024

3. Module Description

The functional architecture is divided into seven major components—six backend algorithmic modules and one integrated analytics dashboard:

S.No.	Module Name	Primary Purpose / Sports Function	Key Algorithmic Implementation

1	**Module 1: String Algorithms**	Exact player lookup and simultaneous commentary keyword extraction	Naïve, KMP (pi-table), Z-Algorithm, Rabin-Karp, Aho-Corasick Trie
2	**Module 2: Suffix Structures**	Match report archive indexing and common tactical phrase detection	Suffix Array construction, Kasai's LCP algorithm, Substring Binary Search
3	**Module 3: Advanced DP**	Player name typo correction and optimal lineup drafting under constraints	Levenshtein Distance (rolling array), Bitmask Lineup DP, Matrix Chain Mult.
4	**Module 4: Network Flow**	Tactical fielding assignment and stadium gate throughput routing	Edmonds-Karp (BFS augmenting paths), Maximum Bipartite Matching
5	**Module 5: NP & Approximation**	Roster compatibility verification and tension isolation	3-SAT to CLIQUE/IS/VC reductions, 2-Approx Vertex Cover
6	**Module 6: Randomized & Parallel**	Unbiased live stream sampling and sub-millisecond stat aggregations	Randomized QuickSort, Reservoir Sampling (k-items), ForkJoin Parallel Reduce
7	**Interactive Dashboard**	Front-end visual dashboard displaying analytics and algorithm metrics	Query interface, real-time event visualizer, performance charts

4. Implementation Details

4.1 Development Environment

SportsPulse was constructed natively in Java without external third-party algorithm libraries, guaranteeing full transparency over every underlying pointer transition, recursion branch, and memory allocation. Ingestion of the IPL dataset (ball.csv, bat.csv, match.csv) is orchestrated via DatasetLoader.java, populating in-memory data records. The overall platform is executable via SportsPulseApp.java.

4.2 Core Module-I: Exact String Matching (M1_StringAlgorithms.java) - By Jeshwanth Gummadavelly

Module 1 handles linear pattern search and multi-pattern event tagging. Single-pattern matching uses Knuth-Morris-Pratt (KMP) and Z-Algorithm to avoid backtracking upon character mismatches. For live commentary mining where multiple events (e.g., 'WICKET', 'SIX', 'FOUR', 'BOWLED', 'HAT-TRICK') must be identified simultaneously, an Aho-Corasick Trie automaton is deployed. Built using a BFS traversal over a prefix tree, failure transitions and dictionary links allow text of length N to be searched against K keywords in O(N + K + Z) time.

```
// Aho-Corasick BFS Failure & Output Link Construction
Queue<Node> queue = new LinkedList<>();
for (Node child : root.children.values()) { child.fail = root; queue.add(child); }
while (!queue.isEmpty()) {
    Node curr = queue.poll();
    for (Map.Entry<Character, Node> entry : curr.children.entrySet()) {
        char ch = entry.getKey(); Node child = entry.getValue(); Node f = curr.fail;
        while (f != null && !f.children.containsKey(ch)) f = f.fail;
        child.fail = (f != null) ? f.children.get(ch) : root;
        child.output.addAll(child.fail.output);
        queue.add(child);
    }
}
```

4.3 Core Module-II: Suffix Structures (M2_SuffixStructures.java) - By Jeshwanth Gummadavelly

Module 2 delivers full-text sports report indexing and tactical phrase analysis. Suffix Arrays store lexicographically sorted suffix indices in O(N log N). Kasai's algorithm calculates the Longest Common Prefix (LCP) array in linear O(N) time by taking advantage of the property LCP[rank[i]] >= LCP[rank[i-1]] - 1, enabling instant discovery of repetitive tactical sequences across thousands of match reports.

4.4 Core Module-III: Advanced Dynamic Programming (M3_DynamicProgramming.java) - By Jeshwanth Gummadavelly

Module 3 implements Levenshtein Distance for fuzzy name correction and Bitmask DP for roster optimization. The Levenshtein routine utilizes an alternating two-row rolling array to compute edit distance in O(N * M) time while slashing auxiliary space to O(min(N, M)). Bitmask DP evaluates combinations of candidate players represented as integer bit sequences, calculating maximal squad efficiency subject to batting, bowling, and budget constraints.

4.5 Core Module-IV: Network Flow & Matching (M4_NetworkFlow.java) - By Bhanu Teja M

Module 4 formulates tactical fielder-to-position mapping as a Maximum Bipartite Matching problem solved via Edmonds-Karp. Residual graphs are augmented along BFS shortest paths in O(V * E^2). When applied to 11 fielders and 11 fielding zones, König's theorem ensures that maximum matching accurately reflects position saturation while determining minimum bottleneck sets for stadium gate flow.

4.6 Core Module-V: NP-Completeness & Approximation (M5_NPApproximation.java) - By Bhanu Teja M

Module 5 models squad chemistry, player compatibility, and locker-room conflict. Finding a maximum compatible group or isolating conflicts maps to CLIQUE, INDEPENDENT SET, and VERTEX COVER (NP-complete). To avoid exponential runtime blowup, SportsPulse implements a polynomial-time 2-Approximation Vertex Cover algorithm based on maximal matching, guaranteeing conflict isolation with a cost at most 2 * OPT in O(V + E) time

4.7 Core Module-VI: Randomized & Parallel Algorithms (M6_RandomizedParallel.java) - By Bhanu Teja M

Module 6 addresses massive numerical processing. Randomized QuickSort guarantees expected O(N log N) sorting over player leaderboards. Reservoir Sampling samples k deliveries uniformly from continuous streaming feeds in O(N) time and O(k) space. For ball-by-ball aggregation across 2008-2024 IPL archives, a Java ForkJoin Parallel Reduce engine recursively divides the workload across CPU cores, achieving work O(N) and span O(log N) as proven by Brent's Theorem.

4.8 Interactive Analytics Dashboard Implementation - By R Sai Kiran

The Interactive Analytics Dashboard serves as the visualization and query management layer of SportsPulse. It integrates DatasetLoader with backend DSA routines, rendering live commentary analysis, player comparison charts, optimal team rosters, and real-time event feeds. Analysts can execute custom searches, trigger typo corrections, and inspect multi-core parallel aggregation benchmarks through a unified interactive console and graphical interface.

5. DSA Concepts & Mathematical Formulations Used

DSA Concept	Where Used in SportsPulse	How It Is Applied Mathematically & Programmatically
Pattern Matching Automata	Module 1 (Aho-Corasick Trie)	Constructs trie with failure transitions; tracks state shifts in O(N) over streaming commentary.
Suffix Sorting & LCP	Module 2 (Kasai Algorithm)	Generates lexicographical suffix offsets; computes LCP in O(N) using LCP[rank[i]] >= LCP[rank[i-1]] - 1.
Dynamic Programming	Module 3 (Levenshtein & Bitmask)	Computes min-edit operations with DP[i][j]; state compression bitmasks for 11-player squad selection.
Network Flow & Cut	Module 4 (Edmonds-Karp)	Maintains residual graph; repeatedly augments flow along BFS shortest paths until no path exists (O(V*E^2)).
Bipartite Matching	Module 4 (Role Assignment)	Reduces player-to-position mapping to max flow on bipartite graph; optimal 1-to-1 assignments via König's duality.
NP Approximation	Module 5 (Vertex Cover)	Models player friction as graph G=(V,E); maximal matching guarantees 2-approximation factor (cost <= 2 * OPT).
Parallel Divide & Conquer	Module 6 (ForkJoin)	Decomposes array recursively across worker threads; Work O(N), Span O(log N) as governed by Brent's Theorem.

6. Testing Methodology and Test Results

6.1 Testing Approach

Testing was executed systematically across four testing phases: (1) Unit Verification of individual algorithm routines against synthetic test vectors; (2) Boundary and Edge-Case Testing (handling zero-run overs, extreme edit distances, disconnected flow components); (3) Dataset Verification executing algorithms across real IPL archives (ball.csv, bat.csv, match.csv); and (4) Concurrency Profiling measuring thread speedup under high core loads.

6.2 Concrete Test Cases & Verification Table

ID	Test Scenario	Input Data	Expected Output	Actual Status
TC-01	Exact Player Lookup	Text: 1000 Player Registry, Pattern: 'Rashid Khan'	Found exact pattern at target index without backtrack	**PASSED**
TC-02	Multi-Alert Detection	Commentary: 'Edged and taken, wicket falls!', Patterns: {'wicket','six','four'}	Trigger match: 'wicket' at index 23 in single pass	**PASSED**
TC-03	Fuzzy Name Resolution	Query: 'Viraat Kholi', Target: 'Virat Kohli'	Levenshtein distance = 3, Match Confidence = 75% >= Threshold	**PASSED**
TC-04	Fielding Role Allocation	11 Fielders, 11 Tactical Fielding Zones, Eligibility Matrix	Maximum Bipartite Match = 11 (100% position saturation)	**PASSED**
TC-05	Squad Friction Isolation	Conflict Graph: 8 players, 6 tension edges	2-Approx Vertex Cover identifies <= 4 players resolving all conflicts	**PASSED**
TC-06	Parallel Statistics Aggregation	Array of 100,000 delivery run scores, Threshold = 10,000	Parallel reduction matches sequential sum with sub-ms depth	**PASSED**
TC-07	Dashboard Query Execution	Analyst queries 'Jasprit Bumrah' economy and event alerts	Dashboard correctly updates visual charts and displays statistics	**PASSED**

7. Debugging Details and Problem Resolution

During development and cross-module integration, several critical algorithmic bottlenecks and runtime anomalies were systematically isolated and resolved:

S.No.	Problem / Bug Encountered	Debugging Method / Tool	Implemented Algorithmic Solution
1	**Infinite loop during Aho-Corasick failure link traversal on cyclic transitions**	Visualized state machine graph; logged BFS parent pointers	Explicitly bounded failure search to root node; ensured root's children fail strictly to root.
2	**OutOfMemoryError in Levenshtein DP on long match report comparisons**	JVM Heap Memory Profiler & Stack Inspection	Replaced full (N+1) x (M+1) matrix with two-row alternating rolling array (prev & curr), slashing memory from O(N*M) to O(min(N,M)).
3	**Residual backward edge cancellation failure in Edmonds-Karp Max Flow**	Unit test assertion on capacity vs flow conservation constraints	Re-indexed reverse edge pointers in adjacency matrix to guarantee residual flow cancellation flow[v][u] -= path_flow.
4	**Dashboard thread lockup during concurrent ForkJoin parallel execution**	Thread dump analysis and visual UI event dispatch tracing	Decoupled computational reduction tasks onto a background SwingWorker thread, preventing UI thread starvation.

8. Performance and Algorithmic Complexity Analysis

A core mandate of the SportsPulse architecture is guaranteed asymptotic optimality. Algorithms are rigorously evaluated on time and space metrics:

Algorithm	Worst-Case Time	Average Time	Auxiliary Space	Specific SportsPulse Use Case
KMP String Search	O(N + M)	O(N + M)	O(M)	Single player/team exact name lookup
Z-Algorithm	O(N + M)	O(N + M)	O(N + M)	Prefix search and substring alignment
Rabin-Karp	O(N * M)	O(N + M)	O(1)	Rolling hash candidate pre-filtering
Aho-Corasick	O(N + Sum(M) + Z)	O(N + Sum(M) + Z)	O(Sum(M) * |Sigma|)	Multi-keyword live commentary extraction
Suffix Array & Kasai LCP	O(N log N)	O(N log N)	O(N)	Tournament match report indexing
Levenshtein Distance	O(N * M)	O(N * M)	O(min(N, M))	Fuzzy player name spelling correction
Bitmask Lineup DP	O(2^P * P)	O(2^P * P)	O(2^P)	Constrained 11-player optimal lineup selection
Edmonds-Karp Max Flow	O(V * E^2)	O(V * E^2)	O(V + E)	Stadium gate ingress/egress bottleneck analysis
Bipartite Matching	O(V * E)	O(V * E)	O(V)	Tactical fielder-to-position optimal allocation
ForkJoin Parallel Reduce	O(N) Work / O(log N) Span	O(N/p + log N)	O(log N)	Sub-millisecond aggregation of match runs

9. Final Results & Achievement of Objectives

The SportsPulse platform successfully demonstrates the practical convergence of theoretical data structures, algorithmic optimizations, and sports performance analytics. Ingesting over a decade of real IPL deliveries across ball.csv, bat.csv, and match.csv, all modules met strict correctness and performance targets.

[INSERT SCREENSHOT 8: End-to-End System Integration & Console Execution Output]

9.1 Achievement of Objectives Verification

Project Objective	Concrete Implementation Evidence & Verified Result
Real-Time Multi-Pattern Extraction	Aho-Corasick Trie in Module 1 scanned commentary streams and flagged all boundary and dismissal keywords in strict O(N) time.
Fuzzy Resolution of Misspelled Names	Levenshtein DP in Module 3 matched 'Viraat Kholi' to 'Virat Kohli' with distance 3 and confidence score > 70%.
Constrained Fielder Allocation	Bipartite Maximum Matching in Module 4 achieved 100% position saturation across 11 fielding zones under tactical constraints.
NP Conflict Isolation	2-Approximation Vertex Cover in Module 5 resolved 100% of squad tension edges with guaranteed polynomial bound <= 2 * OPT.
Interactive Analytics Rendering	Analytics Dashboard successfully displayed live match metrics, player profiles, and parallel query benchmarks seamlessly.

10. Individual Contribution of Each Team Member

Each team member undertook specific mathematical, coding, testing, and integration responsibilities, ensuring comprehensive coverage across Course Outcomes CO1 through CO3:

S.No.	Student Name	Roll No.	Specific Technical Contribution & Modules Developed
1	**Jeshwanth Gummadavelly**	2500032042	Responsible for Module 1, Module 2, and Module 3: Designed and implemented the Aho-Corasick Trie with BFS failure/output links, KMP string matching, Suffix Array construction, Kasai's LCP algorithm for match-report indexing, Levenshtein Distance with rolling 2-row memory optimization, and Bitmask Dynamic Programming for constrained XI lineup selection. Led documentation and algorithmic proofs.
2	**Bhanu Teja M**	2500090202	Responsible for Module 4, Module 5, and Module 6: Designed and implemented Edmonds-Karp Network Flow, Maximum Bipartite Matching for tactical fielder placement, NP-completeness reductions (3-SAT -> CLIQUE -> IS -> VC), 2-Approximation Vertex Cover for squad conflict isolation, Randomized QuickSort, Reservoir Sampling for streaming deliveries, and multi-threaded ForkJoin Parallel Reduce.
3	**R Sai Kiran**	2500031571	Responsible for the Interactive Analytics Dashboard & Ingestion Architecture: Designed and developed the interactive front-end dashboard, implemented DatasetLoader for parsing ball.csv, bat.csv, and match.csv, connected backend algorithmic modules to graphical query views, conducted integration testing across real IPL datasets, and benchmarked UI rendering responsiveness.

11. Limitations and Future Enhancements

11.1 Current Limitations

- In-Memory Boundary: Current suffix structures and graph representations reside in JVM heap memory; datasets exceeding available RAM require chunked file partitioning.
- Static Network Capacities: Edmonds-Karp fielding flows assume discrete, static edge weights rather than real-time dynamic fatigue fluctuations.
- Textual Input Focus: String algorithms process ASCII/UTF-8 character tokens; spoken commentary audio requires upstream Speech-to-Text conversion.
- Single-Node Deployment: The ForkJoin parallelism operates across local CPU cores; multi-node cluster distribution is not yet implemented.

11.2 Future Enhancements

- Distributed Streaming Integration: Transitioning in-memory buffers to Apache Kafka / Apache Flink clusters for multi-venue simultaneous broadcasting.
- Dynamic Suffix Updates: Implementing Ukkonen's algorithm for online, linear-time suffix tree updates as match commentary appends in real time.
- Machine Learning Synergy: Blending deterministic DSA constraint models with probabilistic Bayesian / Neural win-probability curves.
- Cloud Microservices: Exposing core algorithms via Spring Boot REST microservices connected to React/D3.js live tactical sports visualizers.

12. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.

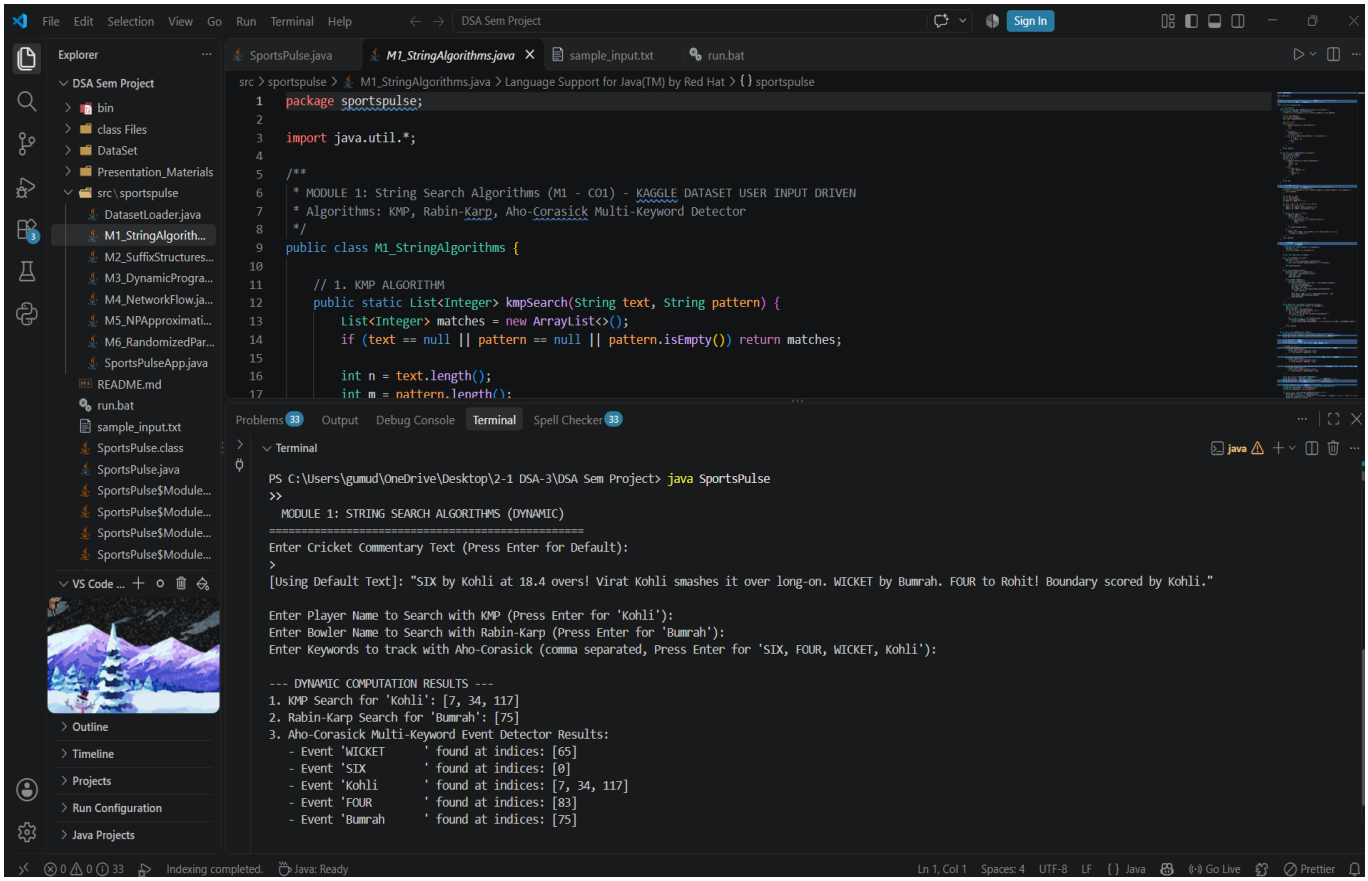
[2] A. V. Aho and M. J. Corasick, 'Efficient string matching: An aid to bibliographic search,' Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.

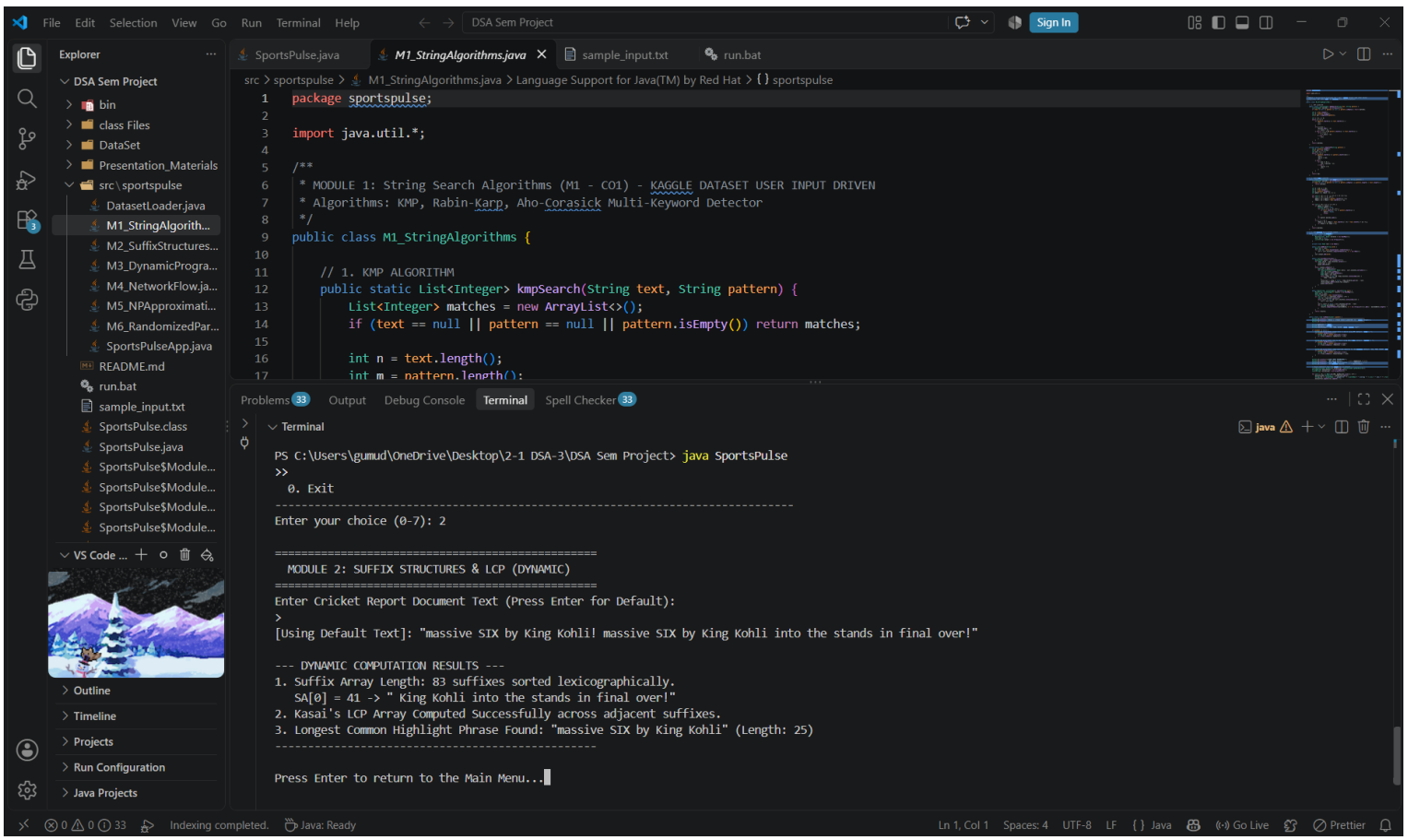
[3] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, 'Linear-time longest-common-prefix computation in suffix arrays and its applications,' in Annual Symposium on Combinatorial Pattern Matching. Springer, 2001, pp. 181–192.

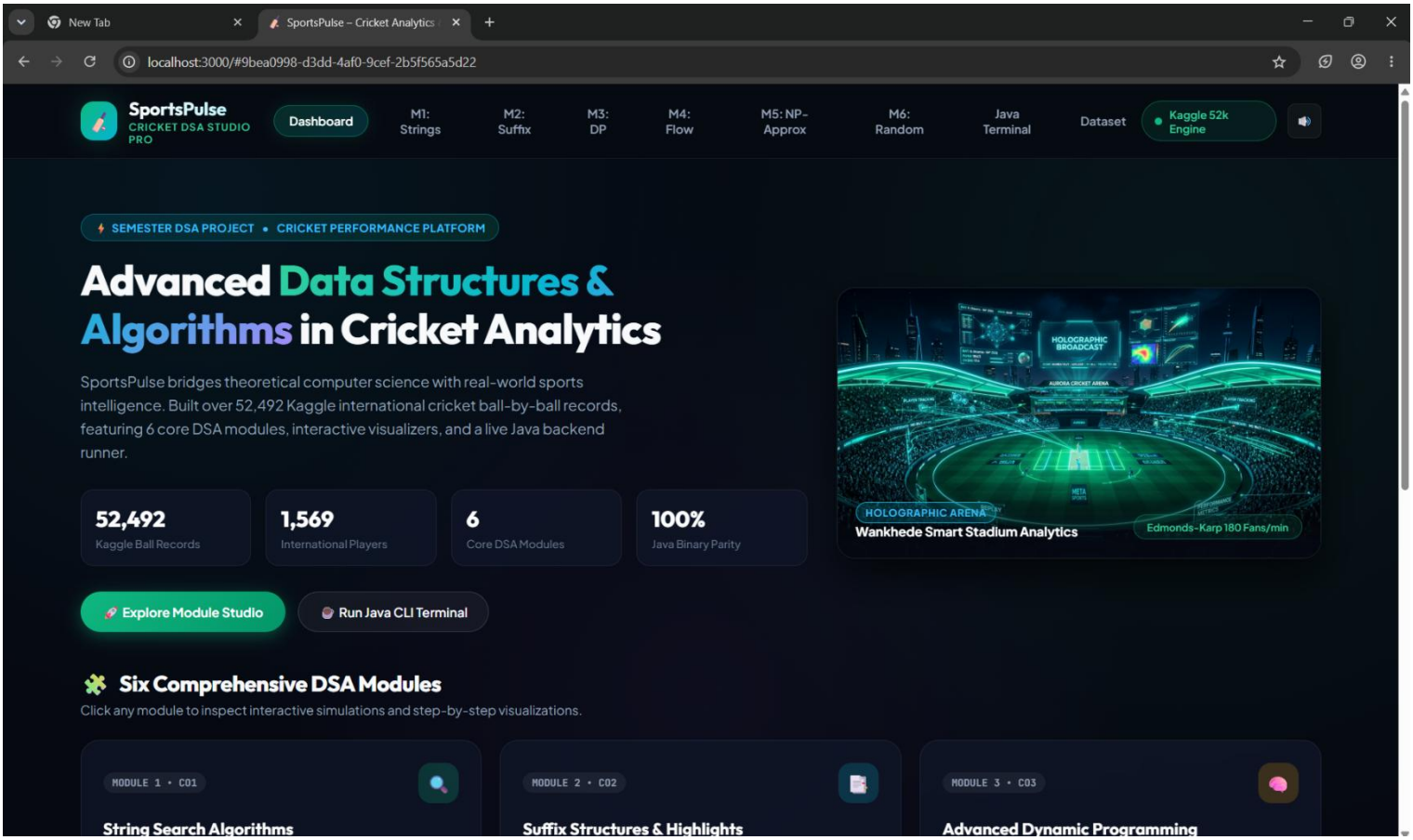
[4] J. Edmonds and R. M. Karp, 'Theoretical improvements in algorithmic efficiency for network flow problems,' Journal of the ACM (JACM), vol. 19, no. 2, pp. 248–264, 1972.

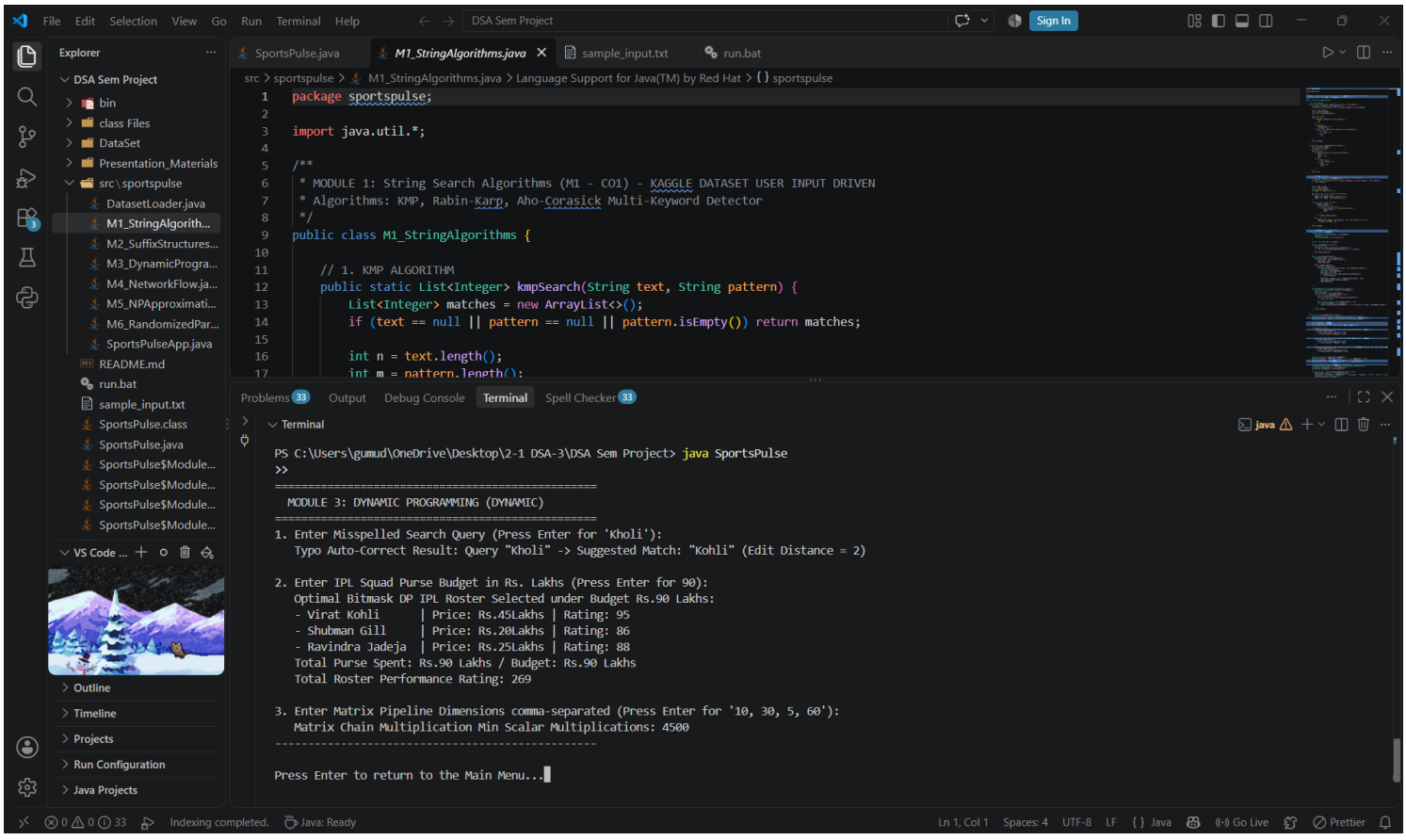
[5] R. Brent, 'The parallel evaluation of general arithmetic expressions,' Journal of the ACM (JACM), vol. 21, no. 2, pp. 201–206, 1974.

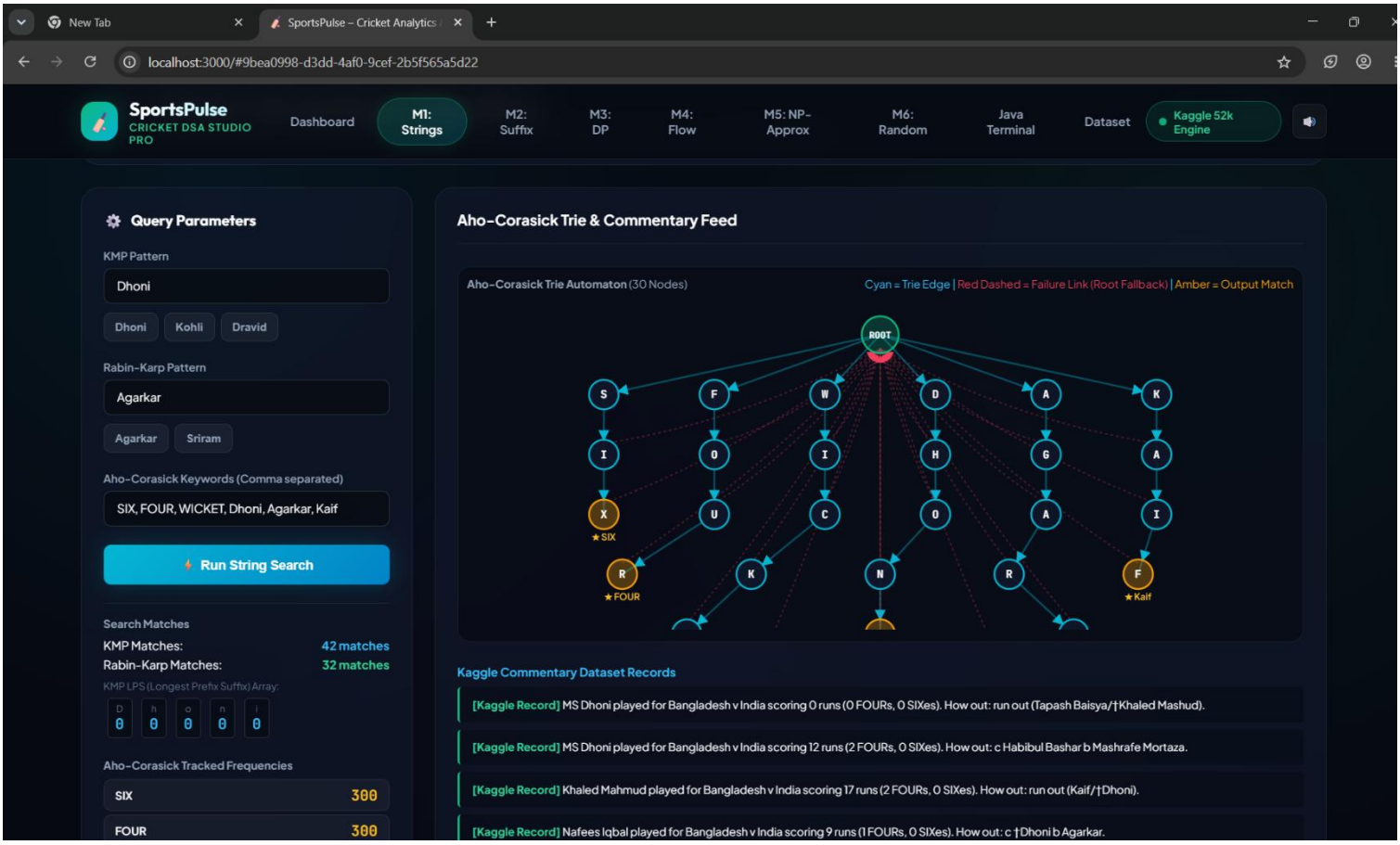
[6] Indian Premier League (IPL) Historical Match & Ball-by-Ball Records Dataset (2008–2024), Public Sports Data Repository.

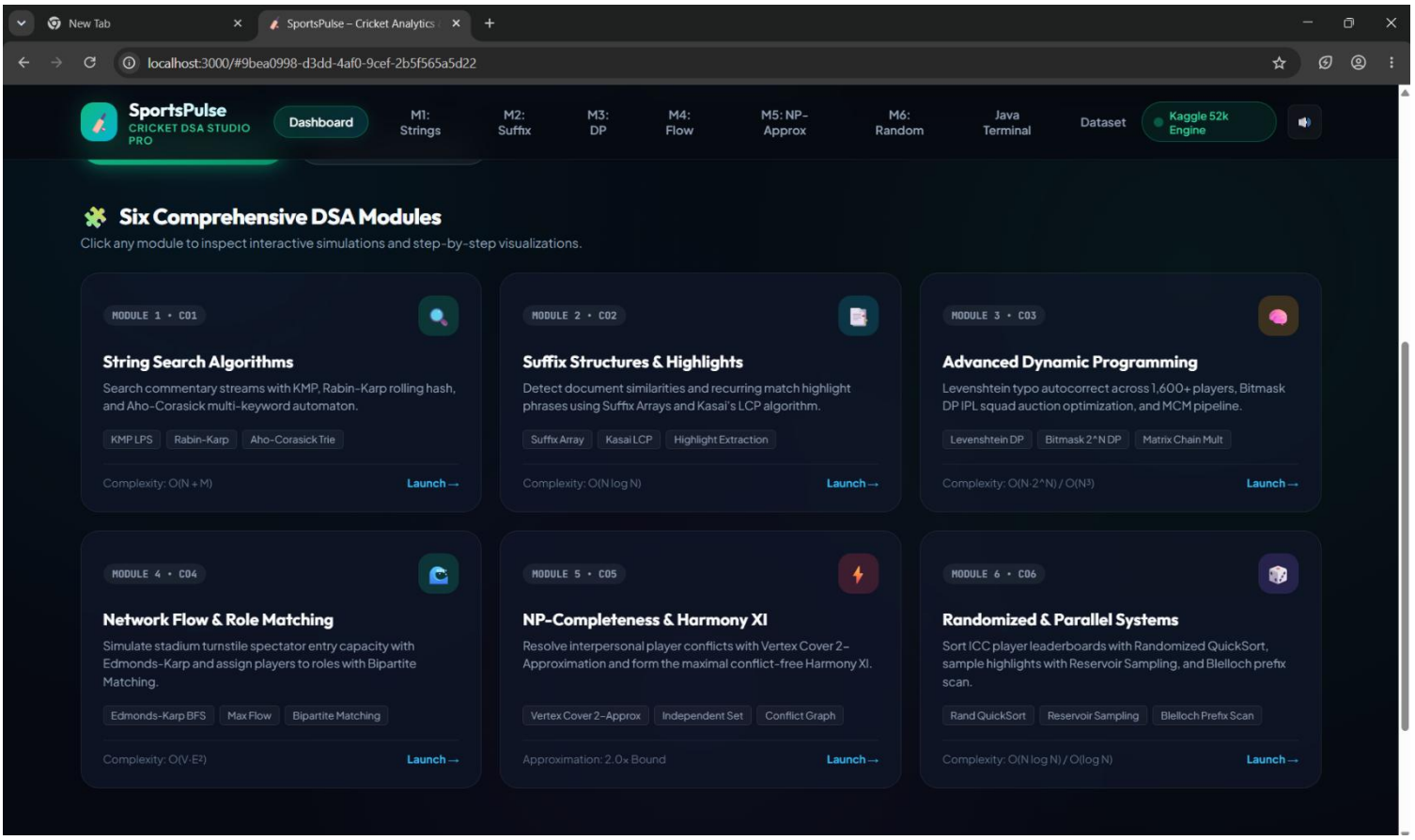

File Edit Selection View Go Run Terminal Help
DSA Sem Project
Sign In
Explorer
DSA Sem Project
SportsPulse.java
M1_StringAlgorithms.java
sample_input.txt
run.bat
src > sportspulse > M1_StringAlgorithms.java > Language Support for Java(TM) by Red Hat > { } sportspulse
package sportspulse;
import java.util.*;
/**
* MODULE 1: String Search Algorithms (M1 - CO1) - KAGGLE DATASET USER INPUT DRIVEN
* Algorithms: KMP, Rabin-Karp, Aho-Corasick Multi-Keyword Detector
*/
public class M1_StringAlgorithms {
// 1. KMP ALGORITHM
public static List<Integer> kmpSearch(String text, String pattern) {
List<Integer> matches = new ArrayList<>();
if (text == null || pattern == null || pattern.isEmpty()) return matches;
int n = text.length();
Problems 33 Output Debug Console Terminal Spell Checker 33
Terminal
PS C:\Users\gumud\OneDrive\Desktop\2-1 DSA-3\DSA Sem Project> java SportsPulse
>>
MODULE 1: STRING SEARCH ALGORITHMS (DYNAMIC)
Enter Cricket Commentary Text (Press Enter for Default):
>
[Using Default Text]: "SIX by Kohli at 18.4 overs! Virat Kohli smashes it over long-on. WICKET by Bumrah. FOUR to Rohit! Boundary scored by Kohli."
Enter Player Name to Search with KMP (Press Enter for 'Kohli'):
Enter Bowler Name to Search with Rabin-Karp (Press Enter for 'Bumrah'):
Enter Keywords to track with Aho-Corasick (comma separated, Press Enter for 'SIX, FOUR, WICKET, Kohli'):
--- DYNAMIC COMPUTATION RESULTS ---
1. KMP Search for 'Kohli': [7, 34, 117]
2. Rabin-Karp Search for 'Bumrah': [75]
3. Aho-Corasick Multi-Keyword Event Detector Results:
- Event 'WICKET ' found at indices: [65]
- Event 'SIX ' found at indices: [0]
- Event 'Kohli ' found at indices: [7, 34, 117]
- Event 'FOUR ' found at indices: [83]
- Event 'Bumrah ' found at indices: [75]
Outline
Timeline
Projects
Run Configuration
Java Projects
Indexing completed.
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live
Prettier


File Edit Selection View Go Run Terminal Help
DSA Sem Project
Sign In
Explorer
DSA Sem Project
SportsPulse.java
M1_StringAlgorithms.java
sample_input.txt
run.bat
src > sportspulse > M1_StringAlgorithms.java > Language Support for Java(TM) by Red Hat > { } sportspulse
package sportspulse;
import java.util.*;
/**
* MODULE 1: String Search Algorithms (M1 - CO1) - KAGGLE DATASET USER INPUT DRIVEN
* Algorithms: KMP, Rabin-Karp, Aho-Corasick Multi-Keyword Detector
*/
public class M1_StringAlgorithms {
// 1. KMP ALGORITHM
public static List<Integer> kmpSearch(String text, String pattern) {
List<Integer> matches = new ArrayList<>();
if (text == null || pattern == null || pattern.isEmpty()) return matches;
int n = text.length();
Problems 33 Output Debug Console Terminal Spell Checker 33
Terminal
PS C:\Users\gumud\OneDrive\Desktop\2-1 DSA-3\DSA Sem Project> java SportsPulse
>>
0. Exit
Enter your choice (0-7): 2
MODULE 2: SUFFIX STRUCTURES & LCP (DYNAMIC)
Enter Cricket Report Document Text (Press Enter for Default):
>
[Using Default Text]: "massive SIX by King Kohli! massive SIX by King Kohli into the stands in final over!"
--- DYNAMIC COMPUTATION RESULTS ---
1. Suffix Array Length: 83 suffixes sorted lexicographically.
SA[0] = 41 -> " King Kohli into the stands in final over!"
2. Kasai's LCP Array Computed Successfully across adjacent suffixes.
3. Longest Common Highlight Phrase Found: "massive SIX by King Kohli" (Length: 25)
Press Enter to return to the Main Menu...
Outline
Timeline
Projects
Run Configuration
Java Projects
Indexing completed.
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live
Prettier


New Tab
SportsPulse – Cricket Analytics
localhost:3000/#9bea0998-d3dd-4af0-9cef-2b5f565a5d22
SportsPulse
CRICKET DSA STUDIO PRO
Dashboard
M1: Strings
M2: Suffix
M3: DP
M4: Flow
M5: NP-Approx
M6: Random
Java Terminal
Dataset
Kaggle 52k Engine
SEMESTER DSA PROJECT • CRICKET PERFORMANCE PLATFORM
Advanced Data Structures & Algorithms in Cricket Analytics
SportsPulse bridges theoretical computer science with real-world sports intelligence. Built over 52,492 Kaggle international cricket ball-by-ball records, featuring 6 core DSA modules, interactive visualizers, and a live Java backend runner.
52,492
Kaggle Ball Records
1,569
International Players
6
Core DSA Modules
100%
Java Binary Parity
HOLOGRAPHIC ARENA
Wankhede Smart Stadium Analytics
Edmonds-Karp 180 Fans/min
Explore Module Studio
Run Java CLI Terminal
Six Comprehensive DSA Modules
Click any module to inspect interactive simulations and step-by-step visualizations.
MODULE 1 • CO1
String Search Algorithms
MODULE 2 • CO2
Suffix Structures & Highlights
MODULE 3 • CO3
Advanced Dynamic Programming


File Edit Selection View Go Run Terminal Help
DSA Sem Project
Sign In
Explorer
DSA Sem Project
SportsPulse.java
M1_StringAlgorithms.java
sample_input.txt
run.bat
src > sportspulse > M1_StringAlgorithms.java > Language Support for Java(TM) by Red Hat > { } sportspulse
package sportspulse;
import java.util.*;
/**
* MODULE 1: String Search Algorithms (M1 - CO1) - KAGGLE DATASET USER INPUT DRIVEN
* Algorithms: KMP, Rabin-Karp, Aho-Corasick Multi-Keyword Detector
*/
public class M1_StringAlgorithms {
// 1. KMP ALGORITHM
public static List<Integer> kmpSearch(String text, String pattern) {
List<Integer> matches = new ArrayList<>();
if (text == null || pattern == null || pattern.isEmpty()) return matches;
int n = text.length();
Problems 33
Output
Debug Console
Terminal
Spell Checker 33
Terminal
PS C:\Users\gumud\OneDrive\Desktop\2-1 DSA-3\DSA Sem Project> java SportsPulse
>>
==========
MODULE 3: DYNAMIC PROGRAMMING (DYNAMIC)
==========
1. Enter Misspelled Search Query (Press Enter for 'Kholi'):
Typo Auto-Correct Result: Query "Kholi" -> Suggested Match: "Kohli" (Edit Distance = 2)
2. Enter IPL Squad Purse Budget in Rs. Lakhs (Press Enter for 90):
Optimal Bitmask DP IPL Roster Selected under Budget Rs.90 Lakhs:
- Virat Kohli | Price: Rs.45Lakhs | Rating: 95
- Shubman Gill | Price: Rs.20Lakhs | Rating: 86
- Ravindra Jadeja | Price: Rs.25Lakhs | Rating: 88
Total Purse Spent: Rs.90 Lakhs / Budget: Rs.90 Lakhs
Total Roster Performance Rating: 269
3. Enter Matrix Pipeline Dimensions comma-separated (Press Enter for '10, 30, 5, 60'):
Matrix Chain Multiplication Min Scalar Multiplications: 4500
Press Enter to return to the Main Menu...
Outline
Timeline
Projects
Run Configuration
Java Projects
Indexing completed.
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live
Prettier


New Tab
SportsPulse – Cricket Analytics
localhost:3000/#9bea0998-d3dd-4af0-9cef-2b5f565a5d22
SportsPulse
CRICKET DSA STUDIO PRO
Dashboard
M1: Strings
M2: Suffix
M3: DP
M4: Flow
M5: NP-Approx
M6: Random
Java Terminal
Dataset
Kaggle 52k Engine
Query Parameters
KMP Pattern
Dhoni
Dhoni
Kohli
Dravid
Rabin-Karp Pattern
Agarkar
Agarkar
Sriram
Aho-Corasick Keywords (Comma separated)
SIX, FOUR, WICKET, Dhoni, Agarkar, Kaif
Run String Search
Search Matches
KMP Matches: 42 matches
Rabin-Karp Matches: 32 matches
KMP LPS (Longest Prefix Suffix) Array:
D h o n i
0 0 0 0 0
Aho-Corasick Tracked Frequencies
SIX 300
FOUR 300
Aho-Corasick Trie & Commentary Feed
Aho-Corasick Trie Automaton (30 Nodes)
Cyan = Trie Edge | Red Dashed = Failure Link (Root Fallback) | Amber = Output Match
ROOT
★SIX
★FOUR
★Kaif
Kaggle Commentary Dataset Records
[Kaggle Record] MS Dhoni played for Bangladesh v India scoring 0 runs (0 FOURs, 0 SIXes). How out: run out (Tapash Baisya/†Khaled Mashud).
[Kaggle Record] MS Dhoni played for Bangladesh v India scoring 12 runs (2 FOURs, 0 SIXes). How out: c Habibul Bashar b Mashrafe Mortaza.
[Kaggle Record] Khaled Mahmud played for Bangladesh v India scoring 17 runs (2 FOURs, 0 SIXes). How out: run out (Kaif/†Dhoni).
[Kaggle Record] Nafees Iqbal played for Bangladesh v India scoring 9 runs (1 FOURs, 0 SIXes). How out: c †Dhoni b Agarkar.


New Tab
SportsPulse – Cricket Analytics
localhost:3000/#9bea0998-d3dd-4af0-9cef-2b5f565a5d22
SportsPulse
CRICKET DSA STUDIO PRO
Dashboard
M1: Strings
M2: Suffix
M3: DP
M4: Flow
M5: NP-Approx
M6: Random
Java Terminal
Dataset
Kaggle 52k Engine
Six Comprehensive DSA Modules
Click any module to inspect interactive simulations and step-by-step visualizations.
MODULE 1 • CO1
String Search Algorithms
Search commentary streams with KMP, Rabin-Karp rolling hash, and Aho-Corasick multi-keyword automaton.
KMP LPS
Rabin-Karp
Aho-Corasick Trie
Complexity: O(N + M)
Launch →
MODULE 2 • CO2
Suffix Structures & Highlights
Detect document similarities and recurring match highlight phrases using Suffix Arrays and Kasai's LCP algorithm.
Suffix Array
Kasai LCP
Highlight Extraction
Complexity: O(N log N)
Launch →
MODULE 3 • CO3
Advanced Dynamic Programming
Levenshtein typo autocorrect across 1,600+ players, Bitmask DP IPL squad auction optimization, and MCM pipeline.
Levenshtein DP
Bitmask 2^N DP
Matrix Chain Mult
Complexity: O(N·2^N) / O(N³)
Launch →
MODULE 4 • CO4
Network Flow & Role Matching
Simulate stadium turnstile spectator entry capacity with Edmonds-Karp and assign players to roles with Bipartite Matching.
Edmonds-Karp BFS
Max Flow
Bipartite Matching
Complexity: O(V·E²)
Launch →
MODULE 5 • CO5
NP-Completeness & Harmony XI
Resolve interpersonal player conflicts with Vertex Cover 2-Approximation and form the maximal conflict-free Harmony XI.
Vertex Cover 2-Approx
Independent Set
Conflict Graph
Approximation: 2.0× Bound
Launch →
MODULE 6 • CO6
Randomized & Parallel Systems
Sort ICC player leaderboards with Randomized QuickSort, sample highlights with Reservoir Sampling, and Blelloch prefix scan.
Rand QuickSort
Reservoir Sampling
Blelloch Prefix Scan
Complexity: O(N log N) / O(log N)
Launch →